

Diseño y análisis de algoritmos

Ejercicios de práctica (DyC, DP)

1. Picos en un vector. Dado un vector u , un **elemento pico** es un elemento que no es menor que sus vecinos inmediatos.
 - a) Encuentre un algoritmo eficiente (mejor que $\mathcal{O}(n)$) que encuentre algún elemento pico en un vector de n posiciones. Observe que puede haber varios elementos pico; basta encontrar uno de ellos (siempre hay por lo menos uno.)
 - b) Calcule la complejidad de su algoritmo y justifique que cumple con el requerimiento.
2. Picos estrictos en un vector. Dado un vector u de números enteros, decimos que un elemento $u[i]$ es un **pico estricto** en u si $u[i-1] < u[i] > u[i+1]$ en el caso de los elementos extremos, $u[0]$ es un pico estricto si $u[0] > u[1]$ y $A[\text{len}(A)-1]$ es un pico estricto si $u[\text{len}(u)-2] < u[\text{len}(u)-1]$. Asumimos que si el arreglo tiene un único elemento, éste es un pico estricto. Observe que no está en general garantizada la presencia de picos estrictos.

Asumiendo que u contiene números todos diferentes, queremos encontrar con una estrategia eficiente (mejor que $\mathcal{O}(n)$) al menos un pico estricto dentro de este arreglo si es que alguno existe.

 - a) Describa una estrategia para encontrar el pico estricto de un arreglo que cumpla con las condiciones de los párrafos precedentes. ¿Existe siempre un pico estricto en este caso? ¿Por qué o por qué no?
 - b) Escriba el pseudo-código de su solución.
 - c) Determine la complejidad de su algoritmo.
3. En este ejercicio consideramos una función *monótonamente decreciente* $f : \mathbb{N} \rightarrow \mathbb{Z}$, esto es, una función definida sobre los números naturales que devuelve valores enteros, de manera que es $f(i) \geq f(i+1)$). Asumiendo que podemos evaluar f en cualquier punto i en tiempo constante, queremos encontrar $n = \min\{i \in \mathbb{N} | f(i) \leq 0\}$. En otras palabras, queremos encontrar el valor en el que f se vuelve negativa.
 - a) Describa una estrategia que resuelva este problema en tiempo mejor que lineal.
 - b) Escriba el pseudo-código de su solución.
 - c) Calcule la complejidad y muestre que se cumple el requerimiento de costo.
 - d) ¿Puede optimizarse esto si la función es estrictamente decreciente?

4. Tenemos dos números enteros positivos a y b . Queremos calcular a^b , pero nuestro lenguaje de programación carece de la operación potencia. Sólo disponemos de las cuatro operaciones aritméticas básicas.
 - a) Describa una estrategia que resuelva este problema en tiempo mejor que $\mathcal{O}(b)$.
 - b) Escriba el pseudo-código de su solución.
 - c) Calcule la complejidad y muestre que se cumple el requerimiento de costo.
5. Cantidad de veces que aparece un elemento en un arreglo ordenado. Tenemos un arreglo ordenado a y un elemento x que está en el arreglo. Queremos encontrar la frecuencia de x en a (es decir, la cantidad de veces que aparece) de manera eficiente.
 - a) Describa una estrategia que encuentre la frecuencia de x en tiempo mejor que lineal.
 - b) Escriba el pseudo-código de su solución.
 - c) Calcule la complejidad y muestre que se cumple el requerimiento de costo.
6. Frecuencia de elementos en un arreglo ordenado. Tenemos un arreglo a ordenado y queremos obtener una lista $freq$ que contenga una tupla (x, f) por cada elemento x en a , donde f es la frecuencia de a . Por ejemplo, si $a = [1, 1, 2, 3, 3, 3, 3]$ entonces $freq = [(1, 2), (2, 1), (3, 4)]$.
 - a) Describa una estrategia que resuelva este problema en tiempo mejor que lineal. Puede buscar inspiración en el ejercicio anterior.
 - b) Escriba el pseudo-código de su solución.
 - c) Calcule la complejidad y muestre que se cumple el requerimiento de costo.
7. Búsqueda en matrices doblemente ordenadas. Tenemos una matriz cuadrada m doblemente ordenada, es decir una matriz $m \in \mathbb{Z}^{n \times n}$ en la que tanto las filas como las columnas están ordenadas en sentido creciente.

Ejemplo: la siguiente es una matriz doblemente ordenada.

$$\begin{bmatrix} 10 & 20 & 30 & 40 \\ 15 & 25 & 35 & 45 \\ 27 & 29 & 37 & 48 \\ 32 & 33 & 39 & 50 \end{bmatrix}$$

Queremos determinar de manera eficiente si un número x está en m .

- a) Defina una estrategia eficiente para encontrar el número x en una matriz cuadrada doblemente ordenada m . Explique su estrategia. Por “eficiente” entendemos una estrategia de complejidad a lo sumo lineal.
- b) Escriba el pseudo-código de su estrategia.
- c) Determine la complejidad de su algoritmo para verificar que el requerimiento de eficiencia se satisfizo.

Ayuda: en el caso de que el elemento del centro de la matriz sea el número buscado, terminamos. Pero ¿Qué sucede si el elemento central es menor que x ? ¿Dónde deberíamos proseguir la búsqueda? ¿Y si el elemento central fuera mayor que x ?

8. Estamos en el año 2099. Un grupo de infectólogos analiza las curvas de contagios diarios del virus Covid-55 que medio siglo antes había asolado al mundo hasta la llegada de la vacuna senegalesa.

Las cifras correspondientes a cada día del año 2055 para diversos sitios se encuentran organizadas en arreglos de 365 posiciones con la cantidad de contagios diarios. En todos los arreglos se da la misma situación: la curva de contagios crece hasta un punto máximo y a partir de ese punto, comienza a decrecer, a menos que el punto máximo se encuentre al final. En otras palabras, si empieza a decrecer ya no crece más. .

- a) Defina una estrategia eficiente (de costo mejor que lineal) para encontrar el máximo número de contagios en cada vector.
 - b) Escriba el pseudo-código de su solución.
 - c) Calcule la complejidad de su algoritmo y justifique su resultado. Una complejidad sin una justificación correcta no se tendrá en cuenta.
9. Dado un arreglo ordenado con números todos diferentes $a[0..n-1]$, usted quiere determinar si existe un índice i tal que $a[i] = i$.
- a) Defina una estrategia eficiente (de costo mejor que lineal) para encontrar el índice i que cumple $a[i] = i$ o que devuelva -1 si tal índice no existe.
 - b) Escriba el pseudo-código de su solución.
 - c) Calcule la complejidad de su algoritmo y justifique su resultado. Una complejidad sin una justificación correcta no se tendrá en cuenta.
10. El rincón surrealista. Huevos y rascacielos. La UMFm (Universidad Municipal de Fraile Muerto) está planeando un ambicioso experimento para determinar el piso más alto desde el que se puede arrojar un huevo al piso sin que se rompa. Para ello, se parte de las siguientes hipótesis:

- Si un huevo se rompe al ser arrojado desde un piso n , entonces se romperá para cualquier piso superior (es decir, para cualquier piso $m > n$.)
- Si un huevo no se rompe al ser arrojado desde un piso n , entonces tampoco se romperá para ningún piso inferior (es decir, para ningún piso $m < n$.)

El experimento debe llevarse a cabo con el mínimo número de huevos arrojados por expreso pedido del portero del edificio.

- a) Defina una estrategia para determinar el piso máximo desde el que se puede arrojar un huevo sin que se rompa con el mínimo número de huevos arrojados.
 - b) ¿Cuál será la cantidad total de huevos arrojados para un edificio de n pisos?
11. Tenemos un arreglo a de números ordenados. Queremos llenar un BST con los números de a de manera que el BST quede lo más balanceado que sea posible.

Nota: asumimos que tenemos un TDA **BST** equipado con las siguientes operaciones:

- **emptyBST**, que devuelve **true** si el BST está vacío y **false** en caso contrario.
- **leftChild** que nos devuelve el sub-árbol izquierdo, que es también de tipo **BST**.
- **rightChild** que nos devuelve el sub-árbol derecho, que es también de tipo **BST**.
- **addElem** que agrega un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **delElem** que elimina un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **root** que devuelve el elemento de la raíz del BST. Falla si el BST está vacío.

- a) Dé una estrategia para resolver este problema en tiempo lineal.
 - b) Escriba el pseudo-código de su solución y calcule la complejidad correspondiente.
 - c) Calcule el costo de su algoritmo.
12. Tenemos un arreglo a de números que representa el listado *pre-order* de un BST. Queremos reconstruir el BST a partir de este listado.

Nota: asumimos que tenemos un TDA **BST** equipado con las siguientes operaciones:

- **emptyBST**, que devuelve **true** si el BST está vacío y **false** en caso contrario.
- **leftChild** que nos devuelve el sub-árbol izquierdo, que es también de tipo **BST**.
- **rightChild** que nos devuelve el sub-árbol derecho, que es también de tipo **BST**.
- **addElem** que agrega un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **delElem** que elimina un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **root** que devuelve el elemento de la raíz del BST. Falla si el BST está vacío.

- a) Dé una estrategia para resolver este problema en tiempo lineal.
 - b) Escriba el pseudo-código de su solución y calcule la complejidad correspondiente.
 - c) Calcule la complejidad y muestree que nos encontramos dentro de los requerimientos de costo.
13. Vectores rotados. Se tiene un vector ordenado v de n números enteros diferentes rotado k posiciones, con $k \in \{0, \dots, n-1\}$. Esto significa que el elemento que estaba en la posición i pasó a la posición $(i+k) \bmod n$.
- a) Describa la estrategia más eficiente para encontrar los valores máximo y mínimo de v . Este vector ha sido rotado k posiciones (no conocemos k ; sólo sabemos que está entre 0 y $n-1$.)
 - b) Escriba el pseudo-código que realiza su estrategia.
 - c) Justifique que su estrategia es eficiente calculando la complejidad de su programa. Justifique su resultado.
14. El reparto del botín. Dos ladrones robaron equipos del laboratorio de una universidad privada. Los cacos desean dividir los objetos robados en dos grupos tales que la diferencia en valor sea mínima. Por ejemplo, si robaron tres equipos de valor \$998, \$499 y \$1500, la división óptima sería $\{\$998, \$499\}$ y $\{\$1500\}$. Una división exacta es imposible, ya que la suma del valor de todos los equipos robados es impar.
- Podemos asumir que los objetos se encuentran en un arreglo *botín*, donde *botín*[i] nos da el valor del objeto i .
- a) Determine cuál es la técnica más adecuada para resolver este problema y describa una estrategia para resolverlo.
 - b) Escriba el pseudo-código correspondiente.

c) Estime la complejidad de su algoritmo.

Ayuda: puede construirse una matriz que tenga tantas filas como objetos tengamos y tantas columnas como el valor sumado de ellos. Se puede entonces marcar sucesivamente los valores a los que se puede llegar. Por ejemplo, si el arreglo comienza con $[6, 2, \dots]$, en la primera fila se marcará 6, en la segunda fila se marcará 2 y 8, y así sucesivamente. t

15. Árboles isomorfos. Dados dos BST, queremos determinar si son isomorfos. Dos BST son isomorfos si el dibujo es idéntico aunque los valores no lo sean.

Nota: asumimos que tenemos un TDA **BST** equipado con las siguientes operaciones:

- **emptyBST**, que devuelve **true** si el BST está vacío y **false** en caso contrario.
- **leftChild** que nos devuelve el sub-árbol izquierdo, que es también de tipo **BST**.
- **rightChild** que nos devuelve el sub-árbol derecho, que es también de tipo **BST**.
- **addElem** que agrega un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **delElem** que elimina un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **root** que devuelve el elemento de la raíz del BST. Falla si el BST está vacío.

a) Describa una estrategia para resolver este problema.

b) Escriba el pseudo-código de su solución.

c) Determine la complejidad de su algoritmo.

Nota: asumimos que tenemos un TDA **BST** equipado con las siguientes operaciones:

- **emptyBST**, que devuelve **true** si el BST está vacío y **false** en caso contrario.
- **leftChild** que nos devuelve el sub-árbol izquierdo, que es también de tipo **BST**.
- **rightChild** que nos devuelve el sub-árbol derecho, que es también de tipo **BST**.
- **addElem** que agrega un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **delElem** que elimina un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **root** que devuelve el elemento de la raíz del BST. Falla si el BST está vacío.

16. Se tienen dos cadenas de caracteres s_1 y s_2 de longitud m y n respectivamente. Queremos encontrar la máxima sub-cadena común.

Nota: este ejercicio no debe ser confundido con el de la máxima subsecuencia común. Si tenemos las cadenas $s_1 = \text{'SCHOENBERG'}$ y $s_2 = \text{'WEBERN'}$, la máxima subcadena común es 'BER' y la máxima subsecuencia común es 'EBER' .

- a) Determine cuál es la técnica más adecuada para resolver este problema y describa una estrategia para resolverlo.
- b) Escriba el pseudo-código correspondiente.
- c) Estime la complejidad de su algoritmo.

17. Tenemos una mochila de capacidad máxima c y un arreglo a de objetos o tales que cada objeto $o[i]$ tiene un valor $o[i].v$ y un peso $o[i].p$. Queremos colocar objetos en la mochila de manera que se obtenga el máximo valor sin que se exceda el límite de peso de la mochila. Los objetos pueden repetirse cuantas veces lo queramos.

- a) Describa una estrategia para resolver este problema.
- b) Escriba el pseudo-código correspondiente.
- c) Estime la complejidad temporal y espacial de su algoritmo.

18. Diferentes posibilidades para subir una escalera. Un niño sube una escalera con una combinación de saltos de uno, dos o tres escalones. Queremos saber cuántas posibilidades tiene para subir una escalera de n escalones. Llamamos a este número $P(n)$.

Algunos ejemplos:

- Si $n = 1$, sólo hay una posibilidad: subir un escalón, o sea $P(1) = |\{(1)\}| = 1$.
- Si $n = 2$, hay dos posibilidades: subir dos tramos de un escalón o uno de dos: $P(2) = |\{(1, 1), (2)\}| = 2$.
- Si $n = 3$, entonces hay cuatro posibilidades: $\{(1, 1, 1), (1, 2), (2, 1), (3)\}$. Por lo tanto, $P(3) = 4$.
- Si $n = 5$, tenemos que $P(5) = 13$.

- a) Describa una estrategia de programación dinámica para encontrar $P(n)$ para dos parámetros arbitrarios.
- b) Escriba el pseudo-código de su solución.
- c) Determine la complejidad de su algoritmo. Justifique su resultado.

19. Combinaciones de dos n elementos tomados de k . Como usted sabe, el número de combinaciones de n elementos tomados de k (que aquí vamos a denotar $C(n, k)$) es la siguiente:

$$C(n, k) = \frac{n!}{(n-k)!k!}$$

Esta fórmula puede escribirse en términos de resultados anteriores:

$$C(n, k) = C(n-1, k-1) + C(n-1, k)$$

Está claro que no tiene sentido calcular esta fórmula para $k > n$.

Algunos ejemplos:

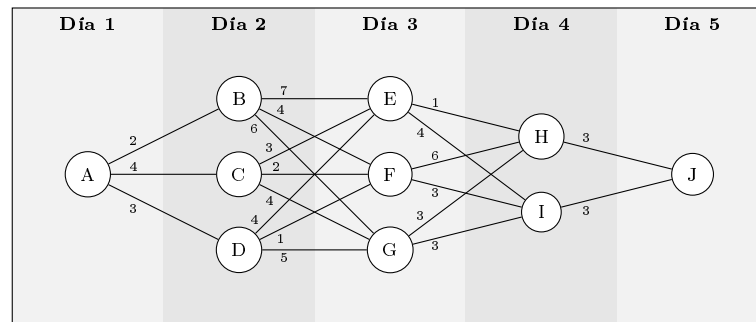
- Si $n = 4$ y $k = 2$, tenemos $C(n, k) = C(4, 2) = \frac{4!}{(4-2)!2!}$. Esto corresponde a los pares $(1, 2), (1, 3), (2, 4), (2, 3), (2, 4), (3, 4)$.
- Si $n = k$ entonces claramente $C(n, k) = C(n, n) = \frac{n!}{0!n!} = 1$.
- Si $k = 0$, entonces $C(n, k) = C(n, 0) = \frac{n!}{n!0!} = 1$.
- Si $n = 4$ y $k = 3$, entonces $C(n, k) = \frac{4!}{(4-3)!3!}$. Esto corresponde a las tuplas $(1, 2, 3), (1, 2, 4), (1, 3, 4), (2, 3, 4)$.

- a) Describa una estrategia para encontrar $C(n, k)$ para dos parámetros arbitrarios n, k con $n \geq k$.
- b) Escriba el pseudo-código de su solución.
- c) Determine la complejidad de su algoritmo. Justifique su resultado.

Ayuda: puede construir una matriz M de $n+1$ filas que vayan de 0 a n y $k+1$ columnas que vayan de 0 a k de manera que en la posición $M[i, j]$ se tenga el coeficiente $C(i, j)$. Las fórmulas más arriba le dan todos los elementos necesarios. Hay que tener en cuenta los casos bases.

20. La diligencia. Un buscador de fortuna de Missouri decide ir al oeste para unirse a la fiebre del oro en California a mediados del siglo XIX. El viaje requiere viajar en diligencia a través de un territorio inexplorado donde existe un grave peligro de ataque. Aunque el punto de partida (Missouri) y el de destino (California) son fijos, hay bastante libertad para elegir qué estados atravesar. Las posibles rutas se muestran en la figura, donde cada estado está representado por un nodo y la dirección del viaje siempre es de izquierda a derecha. Así, hacen falta cuatro etapas (trayectos de diligencia) para viajar desde el punto de embarque en el estado A (Missouri) hasta su destino en el estado J (California).

Esta persona se preocupaba mucho por su seguridad. Tras reflexionar un poco, ideó una forma ingeniosa de determinar la ruta más segura. Se ofrecen pólizas de seguro de vida a los pasajeros de las diligencias. Dado que el costo de la póliza para cualquier viaje en diligencia se basa en una evaluación cuidadosa de la seguridad de dicho viaje, la ruta más segura debería ser aquella con la póliza de seguro de vida total más barata. Estos costos también se muestran en la figura.



- Describa una estrategia para encontrar el camino más seguro.
 - Escriba el pseudo-código de su solución.
 - Determine la complejidad de su algoritmo. Justifique su resultado.
21. Se tiene un BST t y dos números a y b , $a \neq b$, que están en el árbol pero no en la raíz. Queremos un método que nos devuelva el mínimo ancestro común de ambos, es decir el mínimo número que es un ascendiente de a y b . O, si lo prefiere, el mínimo número que tiene en su descendencia a a y a b .

Nota: asumimos que tenemos un TDA **BST** equipado con las siguientes operaciones:

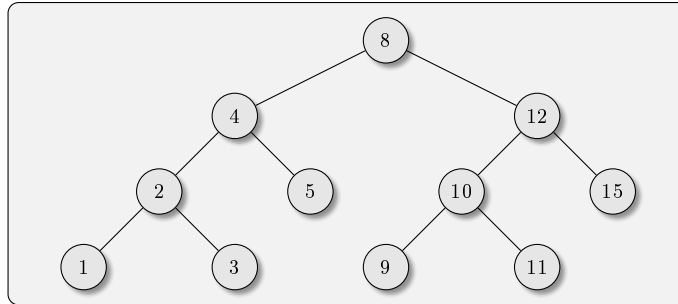
- **emptyBST**, que devuelve **true** si el BST está vacío y **false** en caso contrario.
- **leftChild** que nos devuelve el sub-árbol izquierdo, que es también de tipo **BST**.
- **rightChild** que nos devuelve el sub-árbol derecho, que es también de tipo **BST**.
- **addElem** que agrega un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **delElem** que elimina un elemento al BST con el algoritmo que conocemos de la materia anterior.
- **root** que devuelve el elemento de la raíz del BST. Falla si el BST está vacío.

- Describa la secuencia de pasos que realizará para resolver este ejercicio.

b) Escriba el pseudocódigo correspondiente.

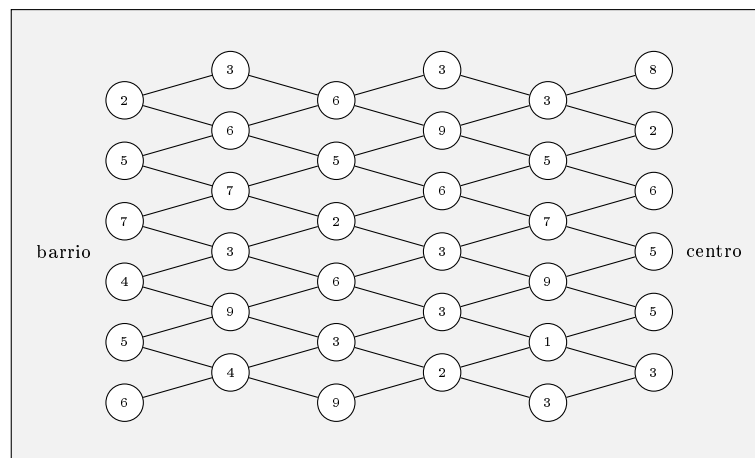
Puede ver un ejemplo en la página siguiente.

Ejemplo: en el árbol de la figura, el programa debería devolver 4 para las entradas 1 y 5, 8 para las entradas 9 y 12, y 8 para las entradas 1 y 11.



Ayuda: ¿cuál es el mínimo ancestro común de dos números cuando uno de ellos es mayor que la raíz?

22. La ciudad de Fraile Muerto tiene un ajetreado centro y el tráfico en horas pico se vuelve lento por las demoras en las esquinas. Queremos encontrar la demora mínima para ir desde el barrio cerrado hasta el centro. Se puede ingresar a por cualquier intersección de la izquierda y se puede salir por cualquier intersección de la derecha.



- a) Describa una estrategia para encontrar el tiempo más rápido para ir desde el barrio hasta el centro.
- b) Escriba el pseudo-código de su solución.

c) ¿Qué debería agregarse para tener además el listado de intersecciones para obtener el mínimo camino? Basta la estrategia. No es necesario el pseudo-código.

23. Se tiene una matriz $M \in \mathbb{N}^{n \times m}$ donde cada posición $M[i, j]$ representa un costo. Un robot se debe desplazar desde la posición $M[0, 0]$ hasta la posición $M[n-1, m-1]$ con mínimo costo. El robot sólo puede desplazarse en sentido horizontal hacia la derecha o vertical hacia abajo; no puede moverse en diagonal. El costo de un camino es la suma de los costos de las celdas visitadas.

a) Describa detalladamente una estrategia de programación dinámica que dé el costo mínimo para ir de $M[0, 0]$ a $M[n-1, m-1]$.

b) Escriba el pseudo-código de su solución y calcule la complejidad correspondiente.

c) Defina una estrategia para recuperar el recorrido de costo mínimo. Se pide sólo la estrategia. No es necesario que haga el pseudo-código.

Ayuda: para el punto **a**, considere la posibilidad, dada una matriz $M \in \mathbb{N}^{n \times m}$ de construir otra matriz $Aux \in \mathbb{N}^{n \times m}$ en la que cada posición $Aux[i, j]$ nos dé el costo mínimo de desplazarse de $M[0, 0]$ a $M[i, j]$.

24. La ciudad de Fraile Muerto está atravesada por el río Ctlamochita. Varios barrios se han construido en ambas márgenes del río y el intendente, en pleno año electoral quiere construir puentes que unan estos barrios. Podemos imaginar esto como un mapa bidimensional con un río horizontal que lo atraviesa por el centro. Hay n bar en la margen sur con coordenadas $x [a_1, a_2, \dots, a_n]$ y n ciudades en la margen norte con coordenadas $x [b_1, b_2, \dots, b_n]$. El intendente quiere conectar la mayor cantidad posible de pares de barrios norte-sur mediante puentes, de manera que ningún puente se cruce con otro. Al conectar las ciudades, sólo podemos conectar la ciudad a_i de la margen norte con la ciudad b_i de la margen sur.

a) Describa una estrategia para encontrar el mayor número de puentes entre ambas márgenes del río.

b) Escriba el pseudo-código de su solución y calcule la complejidad correspondiente.

c) Defina una estrategia para recuperar los barrios que queden conectados por puentes. Se pide sólo la estrategia. No es necesario que haga el pseudo-código.

Ayuda 1: observe para que dos puentes $a_i - b_i$ y $a_j - b_j$ se crucen, es necesario que tengan diferente orden de derecha a izquierda. En otras palabras, debe ser $a_i > a_j$ y $b_i > b_j$ o bien $a_j > a_i$ y $b_j > b_i$.

Ayuda 2: es posible llenar un arreglo p de n posiciones en el que la posición $p[i]$ indique la cantidad máxima de puentes que pueden construirse hasta el punto i .

25. Laberintos. Un *laberinto* es una matriz $\ell \in m \times n$ con una entrada en la posición $\ell[0, 0]$ y una salida en la posición $\ell[m - 1, n - 1]$. En otras palabras, se entra arriba a la izquierda y se sale abajo a la derecha. La matriz tiene únicamente valores 0 y 1. Un valor 0 significa que la posición se puede usar. Un valor 1 significa que hay un obstáculo y que la posición no se puede usar. Sólo se admiten desplazamientos hacia la derecha (este) y hacia abajo (sur). No hacia arriba (norte) ni hacia la izquierda (oeste). Queremos determinar si es posible salir de un laberinto dado.

- Encuentre una estrategia que resuelva el problema.
- Escriba el pseudo-código de su programa.
- Calcule la complejidad de su programa.
- Explique qué debería hacerse para devolver, además de la existencia o no un camino, la secuencia de vértices visitados. No es necesario que escriba el pseudo-código; basta la explicación de la estrategia.

Ayuda: es posible pensar en completar otra matriz indicando si cada posición es accesible o no desde la entrada.

26. El triángulo de Tartaglia. O de Pascal. O de Yang Hui. El coeficiente binomial $C(n, k)$ da el coeficiente de la potencia k de x en el polinomio $(1 + x)^n$. Observe que $C(n, k)$ puede ser computado a partir de resultados previos:

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k)$$

Ejemplos:

- $C(4, 3) = 4$ porque $(1 + x)^4 = 1 + 4x + 6x^2 + \underline{4}x^3 + x^4$.
- $C(4, 3) = 4$ porque $(1 + x)^4 = 1 + 4x + 6x^2 + \underline{4}x^3 + x^4$.

- Describa una estrategia para encontrar $C(n, k)$ a partir de dos valores n y k dados.
- Escriba el pseudo-código de su solución.
- Calcule la complejidad espacial y temporal de su solución.

27. El Centro Ecológico de Fraile Muerto, en el sur de Córdoba, abriga una serie de animales regionales en riesgo de extinción. El cuidador los alimenta con un producto balanceado que cuenta con todos los nutrientes que los animales necesitan. Este alimento viene en latas de 10, 40, y 60 unidades. El cuidador desea minimizar el número de latas que se utilizan (hay que justificar el nombre de “ecológico.”) Se sabe, de acuerdo con el número y tipo de los animales, cuántas unidades de alimento son necesarias.

- a) Describa una estrategia que le permita minimizar las latas para obtener n unidades de alimento.
- b) Escriba el pseudo-código de su algoritmo.
- c) Determine la complejidad espacial y temporal de su algoritmo.
- d) Utilice su algoritmo para determinar la cantidad óptima de 80 y 160 unidades.

28. Ofertas en el supermercado. Un supermercado decide poner en oferta determinados productos. Se decide limitar la cantidad de productos en oferta que pueden ser adquiridos por cliente a dos. Para cada producto en oferta se detalla precio y descuento. Por ejemplo, **aceite de oliva extra virgen, \$ 420.00, 25 %**, lo que significa que si se compra una botella, el ahorro es de **\$ 105.00** y, si se compran dos, el ahorro es de **\$ 210.00**.

Para un cliente hipotético con un presupuesto limitado se desea maximizar el ahorro.

- a) Encuentre la estrategia apropiada para resolver este problema. Describa detalladamente su estrategia (un programador debería saber exactamente qué debe hacer luego de leer su descripción) y diga en cuál de las técnicas vistas en clase se encuadra y por qué. Aplique su estrategia al siguiente conjunto de productos suponiendo que su presupuesto es **\$ 1200.00**.

Producto	Precio	Descuento
A	\$ 500.00	20 %
B	\$ 300.00	25 %
C	\$ 200.00	20 %
D	\$ 400.00	10 %

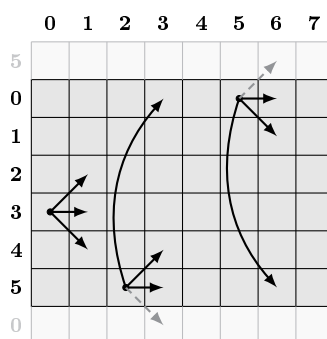
- b) Escriba el pseudo-código que realiza su estrategia.
 - c) Calcule la complejidad de su algoritmo.
29. Caminos de costo mínimo en una grilla cilíndrica. Tenemos una grilla C de m filas y n columnas que ha sido enrollada en forma de cilindro, de manera que la primera y la última fila son adyacentes. Cada columna forma entonces una pista que circunda el cilindro.

Cada posición $C[i, j]$ de la grilla tiene un número entero que se interpreta como un costo. Es posible ir de una pista j a la pista siguiente $j + 1$ pasando de una celda $C[i, j]$ a cualquiera de las celdas adyacentes de la

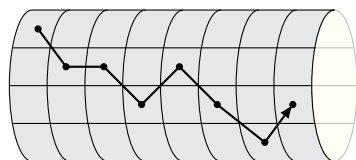
siguiente pista: $C[i + 1, j + 1]$, $C[i, j + 1]$ o $C[i - 1, j + 1]$. Además, dado que la grilla es cilíndrica, es posible pasar de $C[0, j]$ a $C[m - 1, j + 1]$ y de $C[m - 1, j]$ a $C[0, j + 1]$. Éstos son los únicos *desplazamientos válidos*.

Un *recorrido* en una grilla es un camino con desplazamientos válidos desde la pista 0 (primera pista) a la pista $n - 1$ (última pista.) Dado que cada celda contiene un costo, queremos encontrar el recorrido de costo mínimo. Se puede comenzar desde cualquier celda de la primera pista.

Ejemplo: en la siguiente figura se muestran algunos desplazamientos válidos en una grilla cilíndrica de 6 filas por 8 columnas.



A continuación, se muestra un ejemplo de un recorrido en una grilla cilíndrica.



- a) Elabore una estrategia para encontrar el recorrido de menor costo. Recuerde que se puede partir de cualquier fila de la primera pista.
 - b) Escriba el pseudo-código de su solución.
 - c) ¿Qué debe modificarse para que el algoritmo devuelva, además del menor costo, el camino que debe seguirse para lograr este costo?
30. El campo de zanahorias. Se tiene un campo de zanahorias representado por una matriz z , que es una cuadrícula $m \times n$, en la que cada posición $z[i, j]$ representa la cantidad de zanahorias que se pueden encontrar allí. Un conejo recorre el campo de izquierda a derecha. Comienza en la columna 0 y puede elegir cualquier fila desde 0 hasta $m - 1$. Por supuesto, el conejo devorará todas las zanahorias que encuentre en cada posición por la que pase.
- Si el conejo está en la posición $z[i, j]$, tiene las siguientes posibilidades:

- Arriba a la derecha: $z[i, j] \rightarrow z[i - 1, j + 1]$ (si $i > 0$ y $j < m - 1$.)
 - A la derecha: $z[i, j] \rightarrow z[i, j + 1]$ (si $j < m - 1$.)
 - Abajo a la derecha: $z[i, j] \rightarrow z[i + 1, j + 1]$ (si $i < n - 1$ y $j < m - 1$.)
- a) Encuentre una estrategia para que el conejo maximice la cantidad de zanahorias comidas.
 - b) Escriba el pseudo-código de su algoritmo.
 - c) Determine la complejidad espacial y temporal de su algoritmo.